



Institut Supérieur d'Informatique (ISI)

Module : Test des logiciels

RAPPORT TECHNIQUE

ISTQB

Encadré par	Mme Bidan Amani
Réalisé par	Med Talel Kraiem • Med Aziz Bouajina
Année universitaire	2025 – 2026

Introduction

Avant-propos

« Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence. »

— Edsger Dijkstra, The Humble Programmer (1972)

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du module Test des logiciels. Il couvre cinq grandes thématiques complémentaires qui structurent la démarche moderne d'assurance qualité logicielle :

- Les tests unitaires en Java avec JUnit 5, qui valident isolément chaque composant du code.
- Les tests d'intégration avec Cucumber et l'approche BDD (Behavior-Driven Development).
- Le développement piloté par les tests (TDD) appliqué à un cas concret.
- Les tests automatisés avec JUnit et Spring dans une architecture en couches.
- L'utilisation de ChatGPT pour faciliter l'automatisation des tests logiciels.
- Un projet additionnel illustrant la robustesse apportée par une démarche orientée tests.

Chaque chapitre suit une structure identique : présentation de l'objectif, description des outils, mise en œuvre étape par étape, analyse des résultats et synthèse. L'ensemble vise à démontrer que les tests automatisés ne sont pas une étape facultative mais un véritable levier de qualité tout au long du cycle de développement.

Sommaire

- Chapitre 1** Tests Unitaires avec JUnit 5
- Chapitre 2** Test-Driven Development (TDD)
- Chapitre 3** Tests d'Intégration avec Cucumber
- Chapitre 4** Tests automatisés avec JUnit et Spring
- Chapitre 5** ChatGPT pour faciliter l'automatisation des tests
- Chapitre 6** Projet supplémentaire — Liste doublement chaînée
- Conclusion** Synthèse et perspectives

Chapitre 1

Tests Unitaires avec JUnit 5

1. Introduction

Dans ce chapitre, nous mettons en pratique l'écriture de tests unitaires en Java en utilisant les annotations modernes de JUnit 5. Avant de présenter le travail réalisé, il est utile de rappeler la définition d'un test unitaire.

Les tests unitaires sont une pratique de développement logiciel qui consiste à tester chaque composant ou unité individuelle d'un programme de manière isolée et indépendante. L'objectif est de s'assurer que chaque unité fonctionne correctement et produit les résultats attendus, indépendamment du reste de l'application.

2. Outils utilisés

Outil	Description
Eclipse IDE	Environnement de développement intégré puissant et largement utilisé pour le développement Java. Il permet une gestion efficace des projets et une intégration native des outils de test.
JUnit 5	Framework standard pour l'écriture et l'exécution de tests automatisés en Java. Il offre des annotations modernes pour structurer les suites de tests et un système d'assertions riche.

3. Travail réalisé

Au cours de ce TP, nous avons effectué des tests sur la classe `Rational`, composée de deux attributs : numérateur et dénominateur. Cette classe dispose d'un constructeur paramétré, de getters et setters, ainsi que de trois méthodes métier :

- `add` — effectue l'addition de deux nombres rationnels.
- `simplify` — réduit un nombre rationnel à sa forme la plus simple.
- `gcd` — calcule le plus grand commun diviseur entre numérateur et dénominateur.

3.1 Code de la classe `Rational`

```
package proj1;
```

```

public class Rational {
    private int numerator;
    private int denominator;

    public Rational(int numerator, int denominator) {
        if (denominator == 0) {
            throw new IllegalArgumentException("Denominator cannot be zero");
        }
        this.numerator = numerator;
        this.denominator = denominator;
    }

    public int getNumerator() { return numerator; }
    public int getDenominator() { return denominator; }

    public Rational add(Rational other) {
        int newNumerator = (this.numerator * other.denominator)
            + (other.numerator * this.denominator);
        int newDenominator = this.denominator * other.denominator;
        return new Rational(newNumerator, newDenominator);
    }

    public void simplify() {
        int common = gcd(Math.abs(numerator), Math.abs(denominator));
        this.numerator /= common;
        this.denominator /= common;
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }
}

```

3.2 Structure du projet

Comme le montre l'arborescence du projet dans Eclipse, le code source et les classes de test sont volontairement séparés afin de respecter les bonnes pratiques :

```

proj1/
├── src/
│   └── proj1/
│       └── Rational.java
└── test/
    └── proj1/
        ├── CycleDeVie.java
        ├── TestCase1.java
        ├── TestCase2.java
        └── TestTemps.java

```

4. La classe TestCase

Nous avons créé la classe TestCase pour valider la logique métier de nos fractions. Quelques remarques techniques importantes :

- Les méthodes commençant par assert sont des membres statiques de la classe org.junit.jupiter.api.Assertions.
- Grâce à l'import statique import static org.junit.jupiter.api.Assertions.*;, ces méthodes peuvent être appelées directement.
- Si l'assertion est vraie (résultat attendu = résultat constaté), le test continue. Sinon, une exception est levée et le test passe à l'état « échoué » (barre rouge).

5. Tests réalisés et leurs résultats

5.1 Test de la méthode add

La méthode testAddition() vérifie que l'addition de 1/3 et 2/1 donne bien 7/3.

→ **Résultat : test réussi** ✓

5.2 Test de la méthode simplify

La méthode testSimplify() vérifie que les fractions complexes sont correctement réduites à leur forme la plus simple.

→ **Résultat : test réussi** ✓

5.3 Test de déclenchement d'exception (division par zéro)

La méthode testBadDenominator() vérifie que le constructeur lève bien une IllegalArgumentException si le dénominateur fourni est nul.

→ **Résultat : test réussi** ✓

5.4 Imposition d'un temps maximal d'exécution

Dans la classe TimeoutTest, nous utilisons l'annotation @Timeout pour limiter le temps d'exécution d'un test. Si la méthode dépasse le délai imparti (par exemple 10 ms), JUnit l'interrompt et signale un échec.

→ **Résultat : test réussi dans le délai imparti** ✓

6. Analyse des annotations de cycle de vie

Les annotations sont des métadonnées précédées du symbole @. Voici celles utilisées pour gérer le cycle de vie des tests :

Annotation	Rôle
@BeforeAll	Exécute une méthode statique une seule fois avant tous les tests (initialisation globale).
@BeforeEach	S'exécute avant chaque méthode de test (préparation du contexte).
@AfterEach	S'exécute après chaque test (nettoyage des ressources).
@AfterAll	Exécute une méthode statique une fois que tous les tests sont terminés.

6.1 La classe LifeCycleTest

Nous avons créé cette classe afin de visualiser dans la console Eclipse l'ordre exact d'invocation de ces annotations, ce qui aide à mieux comprendre la séquence d'exécution.

7. Analyse et traitement des erreurs

L'un des rôles principaux des tests unitaires est de détecter immédiatement toute régression ou erreur de logique. Pour vérifier l'efficacité de notre suite de tests, nous avons introduit volontairement une erreur dans la méthode testAddition en remplaçant la valeur attendue par 9 au lieu de 7.

Observations dans Eclipse : lors de l'exécution, JUnit signale immédiatement l'échec avec une barre rouge. La vue Failure Trace indique précisément la raison de l'échec :

```
org.opentest4j.AssertionFailedError: expected: <9> but was: <7>
```

Cela confirme que JUnit compare la valeur réelle (7) avec notre attente erronée (9). Cette fonctionnalité permet au développeur de localiser un bug en quelques secondes sans avoir à parcourir manuellement tout le code.

Chapitre 2

Test-Driven Development (TDD)

1. Objectif du TP

L'objectif de ce travail est de mettre en œuvre la méthodologie TDD à travers un exemple concret. Il s'agit de concevoir une classe Loup capable de gérer son orientation (NORD, EST, SUD, OUEST) et de pouvoir tourner dans le sens des aiguilles d'une montre.

2. Principe du TDD

Le Test-Driven Development repose sur un cycle court répété en trois phases :

- RED — écrire un test qui échoue (la fonctionnalité n'existe pas encore).
- GREEN — écrire le minimum de code pour que le test passe.
- REFACTOR — améliorer la qualité du code en gardant tous les tests verts.

3. Mise en œuvre étape par étape

Étape 1 — Test de la position initiale (RED)

Nous commençons par écrire la classe TestLoup pour vérifier que le loup débute à la position NORD.

```
package proj5;

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class TestLoup {
    @Test
    public void testPositionInitialeAuNord() {
        Loup loup = new Loup();
        assertEquals(Orientation.NORD, loup.getOrientation());
    }
}
```

- Constat : erreur de compilation — les classes Loup et Orientation n'existent pas.
- Action : création des squelettes minimaux des classes.
- Résultat : échec du test (barre rouge) avec expected:<NORD> but was:<null>.

Étape 2 — Validation de la position initiale (GREEN)

Nous modifions la classe Loup pour initialiser correctement l'orientation à NORD dans le constructeur.

→ **Résultat : test réussi (barre verte) ✓**

Étape 3 — Rotation du personnage (RED)

Nous ajoutons un nouveau test pour vérifier que le loup tourne correctement vers l'EST.

```
@Test
public void testTournerUneFois() {
    Loup loup = new Loup();
    loup.tourner();
    assertEquals(Orientation.EST, loup.getOrientation());
}
```

- Action : ajout d'une méthode tourner() vide dans Loup.java.

→ **Résultat : échec du test (barre rouge) — le loup reste au NORD.**

Étape 4 — Implémentation de la rotation (GREEN)

Nous écrivons le code de production minimal pour valider le changement d'état.

```
public void tourner() {
    if (this.orientation == Orientation.NORD) {
        this.orientation = Orientation.EST;
    }
}
```

→ **Résultat : test réussi (barre verte) ✓**

Étape 5 — Refactoring et généralisation (REFACTOR)

Pour rendre le code plus évolutif et lisible, nous remanions la méthode tourner() afin de couvrir toutes les directions possibles, en utilisant un switch.

```
public void tourner() {
    switch (orientation) {
        case NORD: orientation = Orientation.EST; break;
        case EST: orientation = Orientation.SUD; break;
        case SUD: orientation = Orientation.OUEST; break;
        case OUEST: orientation = Orientation.NORD; break;
    }
}
```

→ **Résultat : tous les tests restent verts. La structure est désormais robuste et extensible.**

4. Code final de l'implémentation

4.1 Orientation.java

```
package proj5;

public enum Orientation {
    NORD, EST, SUD, OUEST
}
```

4.2 Loup.java

```
package proj5;

public class Loup {
    private Orientation orientation;

    public Loup() {
        this.orientation = Orientation.NORD;
    }

    public void tourner() {
        switch (orientation) {
            case NORD: this.orientation = Orientation.EST; break;
            case EST: this.orientation = Orientation.SUD; break;
            case SUD: this.orientation = Orientation.OUEST; break;
            case OUEST: this.orientation = Orientation.NORD; break;
        }
    }

    public Orientation getOrientation() {
        return orientation;
    }
}
```

Chapitre 3

Tests d'Intégration avec Cucumber

1. Objectif

Les tests d'intégration permettent de vérifier que plusieurs modules d'une application fonctionnent correctement ensemble. Contrairement aux tests unitaires qui isolent chaque unité, les tests d'intégration s'assurent que les interactions entre classes, modules et systèmes externes produisent les résultats attendus.

2. Le Behavior-Driven Development (BDD)

Le BDD est une pratique qui étend le Test-Driven Development en exprimant le comportement attendu de l'utilisateur final dans un langage informel et compréhensible par toutes les parties prenantes (développeurs, testeurs, métier). Ce comportement est ensuite transformé automatiquement en tests exécutables.

3. Cucumber

Cucumber est un framework open source qui permet aux développeurs et testeurs d'écrire des tests automatisés selon l'approche BDD, en utilisant un langage proche du langage naturel appelé Gherkin. Trois mots-clés principaux structurent les scénarios :

- Given — décrit le contexte et les préconditions du test.
- When — décrit l'action ou l'événement déclencheur.
- Then — décrit le résultat attendu.

4. Environnement de travail

Élément	Version / Détail
Système d'exploitation	Linux
IDE	NetBeans + VS Code
JDK	Java 1.8
Gestionnaire de build	Apache Maven

Framework BDD	Cucumber-JVM 1.1.3
Framework de test	JUnit 4.11

5. Architecture du projet

L'architecture standard Maven sépare clairement le code métier (src/main) du code de test (src/test) :

```

project-root/
├── src/
│   ├── main/
│   │   ├── java/          # fichiers source Java
│   │   │   ├── com/
│   │   │   │   └── example/
│   │   │   │       └── MyApp.java
│   │   └── resources/     # ressources non Java
│   └── test/
│       ├── java/          # fichiers de test
│       │   ├── com/
│       │   │   └── example/
│       │   │       └── MyAppTest.java
│       └── resources/     # ressources de test (.feature)
├── target/                # classes compilées
└── pom.xml                # fichier Project Object Model

```

6. Dépendances Maven

Le fichier pom.xml déclare les trois dépendances nécessaires à l'exécution des scénarios Cucumber :

```

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.11</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>info.cukes</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>1.1.3</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>info.cukes</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>1.1.3</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

7. Scénario Gherkin

La syntaxe Gherkin permet de visualiser les comportements attendus de manière lisible :

```
Feature: Fizz Buzz Game

Scenario: Get answers based on played numbers
  Given I am officiating a FizzBuzz game
  When the number 1 is played
  Then I should be told the correct answer is "1"
```

8. Implémentation

8.1 Le runner Cucumber

La classe CukesRunner sert de point d'entrée. Elle est volontairement vide : son rôle se limite à porter les annotations qui déclenchent Cucumber via JUnit.

```
package example.com;

import cucumber.api.junit.Cucumber;
import org.junit.runner.RunWith;

@RunWith(Cucumber.class)
@Cucumber.Options(
    features = "src/test/resources"
)
public class CukesRunner {
}
```

8.2 Les définitions d'étapes (Steps)

La classe ExampleSteps fait le lien entre les phrases Gherkin et le code Java. Chaque méthode est annotée avec @Given, @When ou @Then, suivie d'une expression régulière qui correspond exactement à la phrase du scénario.

```
package example.com;

import cucumber.api.java.en.Given;
import cucumber.api.java.en.When;
import cucumber.api.java.en.Then;
import static org.junit.Assert.assertEquals;

public class ExampleSteps {
    private Example _target;
    private String _actualResult;

    @Given("^I am officiating a FizzBuzz game$")
    public void I_am_officiating_a_FizzBuzz_game() {
        _target = new Example();
    }
}
```

```
}

@When("^the number (\\d+) is played$")
public void the_number_is_played(int playedNumber) {
    _actualResult = _target.checkPlay(playedNumber);
}

@Then("^I should be told the correct answer is \"([^\"]*)\"$")
public void I_should_be_told_the_correct_answer_is(String expected) {
    assertEquals(expected, _actualResult);
}
}
```

9. Résultat de l'exécution

Après lancement de la classe CukesRunner, Cucumber lit le fichier .feature, exécute chacune des étapes et vérifie l'assertion finale. Tous les tests passent avec succès, ce qui se traduit dans l'IDE par une barre verte et le message « All tests passed ».

Chapitre 4

Tests automatisés avec JUnit et Spring

1. Contexte et objectifs

Ce chapitre a pour objectif de mettre en pratique les tests d'intégration dans une architecture Spring en couches. La couche service (logique métier) et la couche DAO (accès aux données) sont strictement séparées, les transactions étant gérées par la couche service.

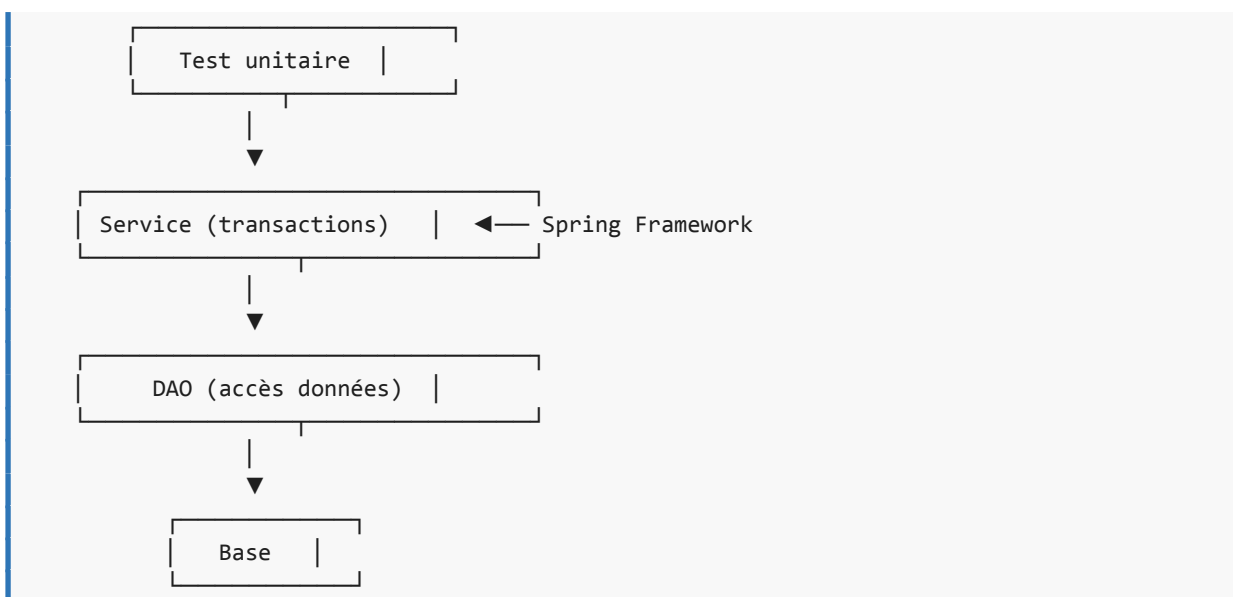
2. Architecture en couches et rôle des transactions

Dans une architecture raisonnable, deux couches sont strictement disjointes :

- La couche de services — contient les traitements métier (règles de gestion, orchestration).
- La couche d'accès aux données (DAO) — contient toutes les requêtes JDBC ou la manipulation des sessions Hibernate.

Dans cette architecture, les transactions sont gérées au niveau de la couche de services. Avec Spring Framework, ces transactions peuvent être gérées de façon automatique grâce à des annotations, ou par programmation, avec ou sans Spring Boot.

2.1 Schéma de l'architecture



3. Test des beans de services

Tester les beans de service est la tâche la plus importante dans cette architecture. La classe de test doit pouvoir accéder au bean dans son contexte Spring. Cela ne pose pas de difficulté particulière puisqu'aucun prérequis transactionnel n'est exigé.

3.1 Initialisation par programmation

La classe de test peut initialiser elle-même le contexte Spring et récupérer le bean à tester dans une méthode annotée `@BeforeAll`. L'annotation `@TestInstance(Lifecycle.PER_CLASS)` est nécessaire pour que la méthode `@BeforeAll` puisse être non statique.

```
@TestInstance(Lifecycle.PER_CLASS)
class ProductServiceTest {

    ProductService service;

    @BeforeAll
    void init() {
        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("application-config.xml");
        service = context.getBean(ProductService.class);
    }

    // méthodes de test...
}
```

La suite du test se déroule de façon traditionnelle : une méthode annotée `@Test` par cas testé, structurée selon le patron GIVEN / WHEN / THEN.

```
public class CourseServiceTest {

    @Test
    void getAll_should_work() {
        // GIVEN

        // WHEN
        List<Product> products = service.getAll();

        // THEN
        assertThat(products).isEmpty();
    }
}
```

3.2 Initialisation par annotation

Plutôt que de gérer l'initialisation manuellement, il est possible de demander à Spring d'injecter directement les beans nécessaires. Cette approche est plus concise et plus déclarative.

Deux annotations clés sur la classe de test :

- `@ExtendWith(SpringExtension.class)` — connecte JUnit 5 au support Spring.
- `@ContextConfiguration(locations = {...})` — précise le fichier de contexte Spring à charger.

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(locations = {"/application-config.xml"})
public class ProductServiceTest {

    @Autowired
    ProductService service;

    // méthodes de test...
}
```

L'injection du bean à tester se fait via `@Autowired` ou `@Resource`. Le reste du test reste identique à la technique précédente.

3.3 Initialisation avec Spring Boot

Avec Spring Boot, l'extension JUnit `@ExtendWith` peut être remplacée par l'annotation unique `@SpringBootTest`, qui charge automatiquement le contexte de l'application. Le reste du test demeure identique.

```
@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class ProductServiceBootTest {

    @Autowired
    ProductService service;

    // méthodes de test...
}
```

4. Test des beans de DAO

Le test des beans de DAO pose un problème supplémentaire : celui des transactions. En effet, lorsqu'une méthode de DAO est appelée, une transaction doit être démarrée puis validée (commit) ou annulée (rollback) après l'appel. Cette gestion transactionnelle, normalement assurée par la couche service, doit donc être prise en charge par la classe de test elle-même.

4.1 Transactions par programmation

En accédant au bean `PlatformTransactionManager`, la classe de test peut démarrer et conclure des transactions. Ces transactions engloberont naturellement les requêtes soumises à la base par le composant DAO.

```
@TestInstance(Lifecycle.PER_CLASS)
public class ProductDaoTest {
```

```

PlatformTransactionManager transactionManager;
ProductDao productDao;

@BeforeAll
void initAll() throws Exception {
    // récupération du contexte Spring...
    transactionManager = context.getBean(PlatformTransactionManager.class);
}

@Test
void findByTitle_should_work() {
    // WHEN
    TransactionStatus status = transactionManager.getTransaction(null);
    List<Product> products = productDao.findByTitleLike("TEST");
    transactionManager.commit(status);

    // THEN
}
}

```

4.2 Méthode utilitaire runInTransaction

Pour améliorer la lisibilité du code, on encapsule la logique transactionnelle dans une méthode utilitaire générique `runInTransaction(...)` qui prend en paramètre une expression lambda (`Supplier<T>`) grâce aux apports du JDK 8.

```

@TestInstance(Lifecycle.PER_CLASS)
public class ProductDaoTest {

    <T> T runInTransaction(Supplier<T> supplier) {
        TransactionStatus status = transactionManager.getTransaction(null);
        T result = supplier.get();
        transactionManager.commit(status);
        return result;
    }

    @Test
    void findByTitle_should_work() {
        List<Product> products = runInTransaction(
            () -> productDao.findByTitleLike("TEST")
        );
    }
}

```

4.3 Transactions par annotation

Pour automatiser la gestion des transactions, il suffit d'ajouter l'annotation `@Transactional` sur la classe de test. Spring se charge alors d'ouvrir une transaction avant chaque méthode `@Test` et de la conclure après (par défaut, elle est annulée par rollback à la fin du test, ce qui évite de polluer la base).

```

@ExtendWith(SpringExtension.class)

```

```
@ContextConfiguration(locations = {"application-config.xml"})
@Transactional
public class ProductDaoTest {

    @Autowired
    ProductDao productDao;

    @Test
    void findByTitle_should_work() {
        List<Product> products = productDao.findByTitleLike("TEST");
    }
}
```

4.4 Cas particulier de Spring Boot

Avec Spring Boot, la combinaison `@SpringBootTest` + `@Transactional` offre la solution la plus concise pour tester un DAO :

```
@SpringBootTest
@Transactional
class ProductDaoBootTest {

    @Autowired
    ProductDao productDao;

    // méthodes de test...
}
```

5. Spring vs Spring Boot dans le contexte des tests

Lors de l'écriture des tests avec Spring classique, le développeur doit prendre en charge manuellement le démarrage du contexte Spring en chargeant explicitement un fichier de configuration XML, puis récupérer lui-même chaque bean nécessaire au test. Cette approche, bien que fonctionnelle, alourdit considérablement le code de test et le rend difficile à maintenir.

Spring Boot simplifie radicalement cette démarche. Il suffit d'ajouter l'annotation `@SpringBootTest` sur la classe de test pour que Spring Boot démarre automatiquement tout le contexte de l'application, configure la base de données, et injecte les beans nécessaires via `@Autowired`. De plus, en ajoutant simplement `@Transactional` sur la classe de test, chaque test s'exécute dans sa propre transaction qui est automatiquement annulée après l'exécution, garantissant ainsi que la base de données reste propre entre les tests.

En résumé, là où Spring classique demande des dizaines de lignes de configuration avant d'écrire le moindre test, Spring Boot permet de démarrer immédiatement avec un minimum de code, ce qui le rend particulièrement adapté dans un contexte de tests automatisés.

6. Application Spring Boot — Projet Bank Account

Pour illustrer concrètement l'apport de Spring Boot, nous avons développé un projet Bank Account simple comportant une couche DAO, une couche service et une suite de tests automatisés.

Code source : <https://github.com/imedchaieb/bankaccount-junit-springboot>

6.1 BankAccountDao.java

```
package com.example.demo;

import org.springframework.stereotype.Repository;
import jakarta.persistence.*;
import java.util.List;

@Repository
public class BankAccountDao {

    @PersistenceContext
    private EntityManager em;

    public void save(BankAccount account) {
        em.persist(account);
    }

    public BankAccount findById(Long id) {
        return em.find(BankAccount.class, id);
    }

    public List<BankAccount> findByOwner(String owner) {
        return em.createQuery(
            "SELECT b FROM BankAccount b WHERE b.owner = :owner",
            BankAccount.class)
            .setParameter("owner", owner)
            .getResultList();
    }

    public List<BankAccount> findAll() {
        return em.createQuery(
            "SELECT b FROM BankAccount b",
            BankAccount.class)
            .getResultList();
    }
}
```

6.2 BankAccountServiceTest.java

```
@SpringBootTest
@Transactional
public class BankAccountServiceTest {

    @Autowired
    BankAccountService service;
```

```

@Test
void createAccount_should_work() {
    BankAccount account = service.createAccount("Imed", 1000.0);
    assertNotNull(account.getId());
    assertEquals("Imed", account.getOwner());
    assertEquals(1000.0, account.getBalance());
}

@Test
void createAccount_negative_balance_should_fail() {
    assertThrows(IllegalArgumentException.class, () ->
        service.createAccount("Imed", -500.0)
    );
}

@Test
void deposit_should_increase_balance() {
    BankAccount account = service.createAccount("Imed", 1000.0);
    service.deposit(account.getId(), 500.0);
    assertEquals(1500.0, service.getBalance(account.getId()));
}
}

```

7. Cycle de vie des annotations transactionnelles

Lorsqu'une méthode de test est transactionnelle, le comportement des annotations de cycle de vie JUnit varie selon qu'elles s'exécutent à l'intérieur ou à l'extérieur de la transaction. Spring fournit deux annotations spécifiques pour gérer ce cas.

Annotation	Comportement
@BeforeAll / @AfterAll	S'exécutent en dehors de la transaction (avant/après tous les tests).
@BeforeEach / @AfterEach	S'exécutent dans la même transaction que la méthode @Test.
@BeforeTransaction	Annotation Spring spécifique : s'exécute avant l'ouverture de la transaction.
@AfterTransaction	Annotation Spring spécifique : s'exécute après la clôture de la transaction.

8. Synthèse comparative

Le tableau ci-dessous synthétise les différentes approches étudiées et leur pertinence selon le contexte.

Approche	Initialisation	Transactions	Cas d'usage
Programmatique	Manuelle (@BeforeAll)	Manuelles via TransactionManager	Tests fins / contrôle total
Annotations Spring	@ExtendWith + @ContextConfiguration	@Transactional sur la classe	Tests classiques sans Spring Boot
Spring Boot	@SpringBootTest	@Transactional	Solution la plus concise et moderne

Chapitre 5

ChatGPT pour faciliter l'automatisation des tests logiciels

1. Introduction

L'objectif de ce TP est d'explorer comment ChatGPT peut assister un ingénieur QA dans la création d'un pipeline d'automatisation complet. Nous avons testé la capacité de l'IA à générer du code, à concevoir des scénarios de tests complexes (microservices) et à intégrer des bonnes pratiques de sécurité et de résilience.

2. Automatisation simple — Selenium, Java et Cloud

La première étape consiste à générer un script Selenium capable d'ouvrir Google et d'afficher son titre, tout en s'exécutant sur la grille cloud LambdaTest.

Prompt utilisé

```
Je travaille sur un module de qualité et tests. Peux-tu générer un script Selenium en Java qui ouvre le site google.com, récupère son titre et l'affiche dans la console ? Le code doit être structuré pour être exécutable dans Eclipse et utiliser les identifiants LambdaTest (Username et Access Key) via des variables d'environnement.
```

Analyse du résultat

ChatGPT a généré un code structuré utilisant RemoteWebDriver. Il a fallu configurer les variables d'environnement LT_USERNAME et LT_ACCESS_KEY dans Eclipse pour garantir la sécurité (pas de secrets dans le code source).

3. Conception de tests complexes (architecture microservices)

Nous avons ensuite simulé une plateforme composée de quatre microservices :

- hub_service — point d'entrée des tests.
- test_management_service — validation et préparation.
- linux_container_service — exécution dans des conteneurs.
- log_management_service — collecte et stockage des logs.

3.1 Approche BDD avec Cucumber (Gherkin)

Pour définir le comportement attendu, nous avons demandé à ChatGPT de générer un fichier .feature contenant à la fois un scénario nominal et des scénarios d'échec.

```

Feature: Exécution de tests et gestion des logs
           dans une architecture microservices

Background:
  Given le hub_service est opérationnel
  And le test_management_service est disponible
  And le linux_container_service est disponible
  And le log_management_service est disponible

# Scénario nominal
Scenario: Flux complet du test à la récupération des logs
  Given un utilisateur soumet un test valide au hub_service
  When le hub_service transmet le test au test_management_service
  And le test_management_service valide et prépare le test
  And le test est envoyé au linux_container_service pour exécution
  And le linux_container_service exécute le test avec succès
  And les logs sont envoyés au log_management_service
  Then le log_management_service stocke les logs correctement
  And les logs sont retournés à l'utilisateur

# Scénario négatif 1 – échec test_management_service
Scenario: Échec lors de la validation du test
  Given un utilisateur soumet un test valide au hub_service
  When le test_management_service rencontre une erreur
  Then le hub_service reçoit une erreur
  And l'utilisateur est notifié de l'échec

# Scénario négatif 2 – échec Linux_container_service
Scenario: Échec lors de l'exécution du test
  Given un utilisateur soumet un test valide
  When le linux_container_service échoue à exécuter le test
  Then une erreur d'exécution est renvoyée au hub_service
  And les logs d'erreur sont stockés

```

4. Implémentation technique avec Playwright

Pour automatiser ces scénarios, nous avons sollicité ChatGPT pour générer un script en Playwright (JavaScript). Trois points clés ont été demandés :

- Résilience — utilisation de blocs try/catch pour capturer les erreurs de service.
- Sécurité — utilisation de process.env pour les URL des microservices (pas d'URL en dur).
- Persistance — intégration de logique MySQL pour vérifier la création d'entités en base.

4.1 Extrait du script Playwright

```
import { test, expect, request } from '@playwright/test';
```



```

const HUB_URL      = process.env.HUB_SERVICE_URL;
const TEST_MGMT_URL = process.env.TEST_MANAGEMENT_URL;
const CONTAINER_URL = process.env.LINUX_CONTAINER_URL;
const LOG_URL      = process.env.LOG_MANAGEMENT_URL;

test.describe('Plateforme microservices', () => {
  test('Flux complet', async () => {
    const apiContext = await request.newContext();

    // 1. Soumission du test
    const submit = await apiContext.post(`${HUB_URL}/submit-test`, {
      data: { test: "echo Hello World" }
    });
    expect(submit.ok()).toBeTruthy();
    const { testId } = await submit.json();

    // 2. Vérification via test_management_service
    const tm = await apiContext.get(`${TEST_MGMT_URL}/tests/${testId}`);
    expect(tm.ok()).toBeTruthy();

    // 3. Exécution
    const exec = await apiContext.post(`${CONTAINER_URL}/execute`, {
      data: { testId }
    });
    expect(exec.ok()).toBeTruthy();

    // 4. Récupération des Logs
    const logs = await apiContext.get(`${LOG_URL}/logs/${testId}`);
    const data = await logs.json();
    expect(data.logs).toContain("Hello World");
  });
});

```

5. Orchestration CI/CD et industrialisation

Pour rendre le processus portable et automatique, nous avons demandé à ChatGPT de générer un workflow GitHub Actions exécutant les tests quotidiennement à 9 h ainsi qu'un fichier package.json correspondant.

5.1 Workflow GitHub Actions

```

name: Playwright Tests

on:
  schedule:
    - cron: '0 8 * * *'
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest

```

```
env:
  HUB_SERVICE_URL: ${ secrets.HUB_SERVICE_URL }
  TEST_MANAGEMENT_URL: ${ secrets.TEST_MANAGEMENT_URL }
  LINUX_CONTAINER_URL: ${ secrets.LINUX_CONTAINER_URL }
  LOG_MANAGEMENT_URL: ${ secrets.LOG_MANAGEMENT_URL }
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 20
  - run: npm install
  - run: npx playwright install --with-deps
  - run: npm test
```

5.2 Fichier package.json

```
{
  "name": "microservices-playwright-tests",
  "version": "1.0.0",
  "description": "Tests E2E Playwright",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "report": "playwright show-report"
  },
  "devDependencies": {
    "@playwright/test": "^1.44.0"
  }
}
```

Chapitre 6

Projet supplémentaire — Liste doublement chaînée

1. Motivation

Les listes doublement chaînées sont notoirement délicates à programmer correctement en raison de la manipulation complexe de pointeurs qu'elles impliquent. Nous avons décidé d'appliquer le développement piloté par les tests (TDD) afin de garantir notre code contre les fuites mémoire, les pointeurs pendants et de nombreuses autres erreurs courantes.

2. Méthodes implémentées

Méthode	Description
push_front / push_back	Insertion d'éléments aux deux extrémités de la liste.
pop_front / pop_back	Retrait d'éléments aux deux extrémités de la liste.
locate_node	Retourne la position entière d'un élément dans la liste.
iterators	Permettent une manipulation itérative aisée de la liste.
from	Construction d'une liste à partir d'un autre type.
append	Concatène deux listes en vidant la liste source.
take	Retourne uniquement les N premiers éléments.
resize	Redimensionne la liste sur place pour atteindre une longueur donnée.
contains	Vérifie si la liste contient une valeur spécifique.

3. Approche

Nous avons utilisé la programmation orientée tests pour affiner nos algorithmes en langage Rust, en commençant systématiquement par définir le test avant l'implémentation.

3.1 Test d'insertion (push)

```
#[test]
fn push() {
```

```

let mut list: LinkedList<usize> = LinkedList::new();
assert_eq!(list.push_front_mut(1), &mut 1);
assert_eq!(list.push_back_mut(2), &mut 2);
assert_eq!(list.push_front_mut(0), &mut 0);
assert_eq!(list.push_back_mut(3), &mut 3);
}

```

3.2 Test de retrait (pop)

```

#[test]
fn popers() {
    let mut list: LinkedList<usize> = LinkedList::new();
    list.push_front_mut(1);
    list.push_back_mut(2);
    list.push_front_mut(0);
    list.push_back_mut(3);

    assert_eq!(list.pop_front(), Some(0));
    assert_eq!(list.pop_back(), Some(3));
    assert_eq!(list.pop_front(), Some(1));
    assert_eq!(list.pop_back(), Some(2));
    assert_eq!(list.pop_front(), None);
    assert_eq!(list.pop_back(), None);
}

```

4. Résultats finaux

Après exécution via l'outil cargo, l'ensemble des 17 tests passent avec succès :

```

$ cargo test

running 17 tests
test linked_list::test::accessors_after_pop ... ok
test linked_list::test::accessors_after_push ... ok
test linked_list::test::contains ... ok
test linked_list::test::extend ... ok
test linked_list::test::accessors_empty ... ok
test linked_list::test::from ... ok
test linked_list::test::locate_node ... ok
test linked_list::test::append ... ok
test linked_list::test::iterators ... ok
test linked_list::test::macros ... ok
test linked_list::test::popers ... ok
test linked_list::test::push ... ok
test linked_list::test::repeat ... ok
test linked_list::test::resize ... ok
test linked_list::test::rotate ... ok
test linked_list::test::take_skip ... ok
test linked_list::test::traits ... ok

test result: ok. 17 passed; 0 failed; 0 ignored

```

→ Une démarche de développement réussie. Le TDD a permis de produire un code à la fois correct et robuste face aux nombreux pièges des structures de données à pointeurs.

Conclusion

Synthèse et perspectives

Ce rapport a couvert les principaux paradigmes de test logiciel modernes : tests unitaires avec JUnit 5, tests d'intégration avec Cucumber et BDD, développement piloté par les tests (TDD), tests automatisés dans une architecture Spring, automatisation assistée par ChatGPT et industrialisation via CI/CD. Chaque approche apporte une couche de garantie complémentaire :

- Les tests unitaires assurent la justesse des composants pris isolément.
- Les tests d'intégration vérifient que les modules collaborent correctement.
- Le TDD guide la conception du code en partant de l'usage attendu.
- Spring et Spring Boot offrent un cadre standard pour tester des applications en couches avec gestion automatique des transactions.
- Le BDD aligne les tests sur le langage métier et facilite la communication avec les parties prenantes.
- L'automatisation et la CI/CD garantissent que cette qualité reste constante au fil des modifications.

La citation de Dijkstra rappelle qu'aucun test ne prouve l'absence totale de bugs ; néanmoins, une stratégie de test rigoureuse et automatisée reste à ce jour le moyen le plus efficace de produire un logiciel fiable, maintenable et conforme aux attentes des utilisateurs.

Les compétences mises en œuvre tout au long de ces travaux pratiques — écriture d'assertions, conception de scénarios Gherkin, application du cycle RED/GREEN/REFACTOR, gestion des transactions Spring, intégration dans un pipeline CI/CD — constituent un socle solide pour la pratique professionnelle de l'ingénierie de la qualité logicielle.